

Universidad de Jaén

Tema 4. Configuración y Despliegue de Aplicaciones en la Nube

Aplicaciones Seguras en la Nube

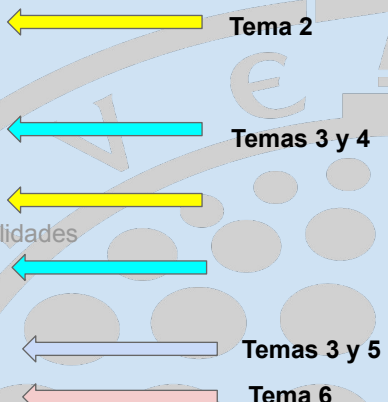
José Ramón Balsas Almagro
Departamento de Informática
Universidad de Jaén
jrbalsas@ujaen.es

Este obra está bajo una Licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

v 1.5.3

La seguridad en el desarrollo software en la nube

- **Análisis:**
  - Identificación de amenazas
  - Análisis de riesgos
  - Requisitos de seguridad: funcionales y no funcionales
- **Diseño**
  - Patrones de diseño seguros en la nube
- **Implementación y despliegue**
  - Buenas prácticas y técnicas de protección
  - Herramientas y bibliotecas de seguridad para minimizar vulnerabilidades
  - Configuración de servicios
- **Seguimiento y evaluación contínua de seguridad**
  - Monitorización y respuesta
  - Evaluación del rendimiento y alta disponibilidad
  - Métodos y Herramientas de verificación de seguridad:
    - Análisis estático de código
    - Análisis dinámico: herramientas de análisis automatizado y *pentesting*



Importancia del despliegue y seguimiento de recursos en la nube

Cuando las órdenes se dan de manera clara
y las tropas están bien organizadas,
el ejército se mueve como un solo hombre

Sun Tzu, El Arte de la Guerra. Siglo V a.C.



Objetivos

- Comprender cómo se **organiza y asegura el despliegue** de una aplicación en la nube
- Saber **configurar un despliegue sencillo** de una aplicación en la nube
- Comprender las **tendencias en gestión y despliegue** de servicios en la nube
- Conocer y saber utilizar los elementos básicos de **Kubernetes** como entorno de **orquestación** de servicios en la Nube

Introducción

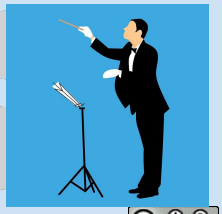
Motivación:

¿Cómo desplegar una aplicación distribuida de forma segura en la nube?

Decisiones:

1. Elección de **tipo de servicios** para componentes: IaaS, PaaS, FaaS, CaaS, SaaS...
2. **Ubicación** de componentes, escalabilidad, alta disponibilidad
3. **Interconexión** de componentes
4. Establecer **restricciones de acceso**
5. **Gestión de cambios**

Orquestación



Retos en despliegue de aplicaciones en la nube

- **Diferentes servicios** y sus características según el proveedor
- **Diferentes interfaces administrativas** en un mismo proveedor: Web, CLI, Api REST
- **Entornos heterogéneos** (nubes híbridas), servicios en redes de empresa... (*on-premises*)
- Dificultad para **realizar, replicar, supervisar despliegues...**¿automatización?

Soluciones:

- (clásica) Aprovisionamiento mediante scripts cli/api rest
- (actual) Sistemas de **orquestación** para control de despliegues
  - Codificación de las características del despliegue de sistemas distribuidos. ***Infraestructura as a Code, IaaC***

¿Cómo organizar el despliegue de una aplicación en la nube?

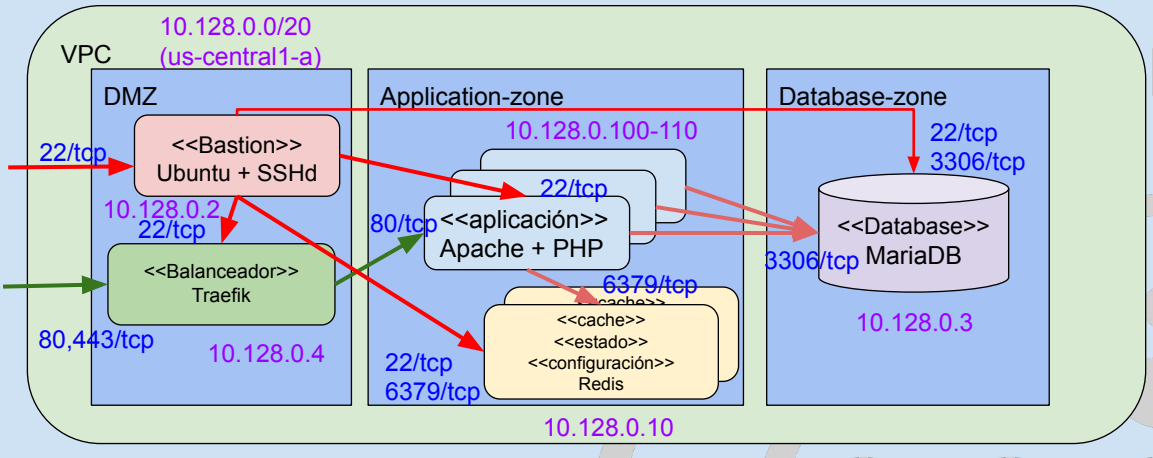
- **Clasificar componentes** atendiendo a su **funcionalidad y grado de interrelación**, e.g. frontales web, servidores de aplicaciones, sistemas de almacenamiento, etc.
- **Organización de la red: segmentación** (HA y seguridad)
  - Regiones geográficas, e.g. GCP us-central1 en Council Bluffs, Iowa
  - Zonas en la región, e.g. us-central1-a, us-central1-b, …
  - Subredes IP públicas y privadas
- **Establecer controles de acceso en la red**:
  - Acceso público desde Internet (DMZ)
  - Acceso entre componentes
  - Acceso de administración



Controles de red en la nube

- **Objetivos:**
  - **Dificultar/contener el acceso no autorizado** a los componentes de la aplicación
  - Facilitar la **detección** de problemas y posibles ataques
  - **Complementar las medidas de seguridad** implantadas en la aplicación
- **Diferencias con redes privadas:**
  - Perímetro de red (DMZ) no definido, e.g. múltiples zonas geográficas, proveedores, servicios SaaS, …
  - Rápido aprovisionamiento y configuración homogénea, e.g. VPC, ACL, Security Groups, …
  - Diferencias y limitaciones según el tipo de servicio: IaaS, PaaS, CaaS, FaaS

Ejemplo de segmentación de red



El sentido de las fechas indica qué conexiones pueden establecerse expresamente para evitar desplazamientos laterales si un host resulta comprometido.
Si no hay flecha → conexión no permitida ¿se deben permitir conexiones hacia el exterior?

Métodos de segmentación de red en la nube

Virtual Private Clouds (VPC)

- Permiten **organizar y aislar** recursos definiendo **subredes privadas** independientes
  - 10.x.x.x, 192.168.x.x, 172.16-31.x.x
  - Independientes de la distribución física de los recursos. Pueden generar gastos por tráfico entre zonas o regiones.
- Las **tecnologías empleadas** y funcionalidades dependen del proveedor o tecnología de orquestación.
  - El acceso público se establece mediante **NAT** (*Network Address Translation*) y/o uso de ***Proxys-Gateways***
- Uso *de listas de control de acceso* para **determinar tráfico entrante/saliente**
- Suelen permitir **registros de actividad**

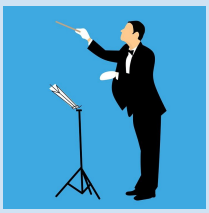
Ejemplo de uso en Anexos

Listas de control de acceso (ACL, *Security Groups, Firewall rules...)*

- Definen reglas de acceso a **grupos de elementos** específicos, e.g. VM, contenedores, servicios, etc.
- Similares a reglas clásicas de cortafuegos en SO: IPs origen, destino, puertos, protocolos, etiquetas, ...

Automatización de despliegues

Orquestación

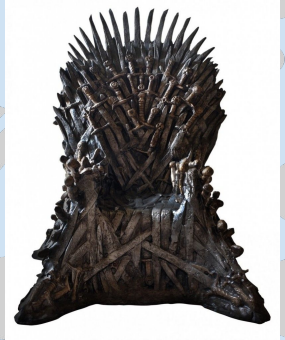


Soluciones de Orquestación

- **Dependientes del proveedor o infraestructura**
  - AWS CloudFormation, CF (vídeo)
    - *"... easy way to create and manage a collection of related AWS resources, provisioning and updating them in an orderly and predictable fashion"*
  - Azure Resource Manager templates, ARM
  - Cloud Deployment Manager, DM, (vídeo)
- **Independientes del proveedor**
  - Openstack
  - Docker compose, Docker swarm
  - Openshift
  - Apache Mesos
  - Hashicorp Terraform (vídeo) → OpenTofu
  - Pulumi
  - ...

¿Una solución para gobernarlas a todas?





Kubernetes, K8s



Definición:

"An open source system for automating deployment, scaling, and management of containerized applications across clusters of hosts"

Historia

- 2014 Google hace open-source su proyecto *Borg* de gestión de contenedores (usado en Gmail o Google Search)
- Cedido a la *Cloud Native Computing Foundation, CNCF*
- Actualmente es ofrecido por los grandes proveedores en la nube: Amazon EKS, Google GKE, Microsoft AKS…

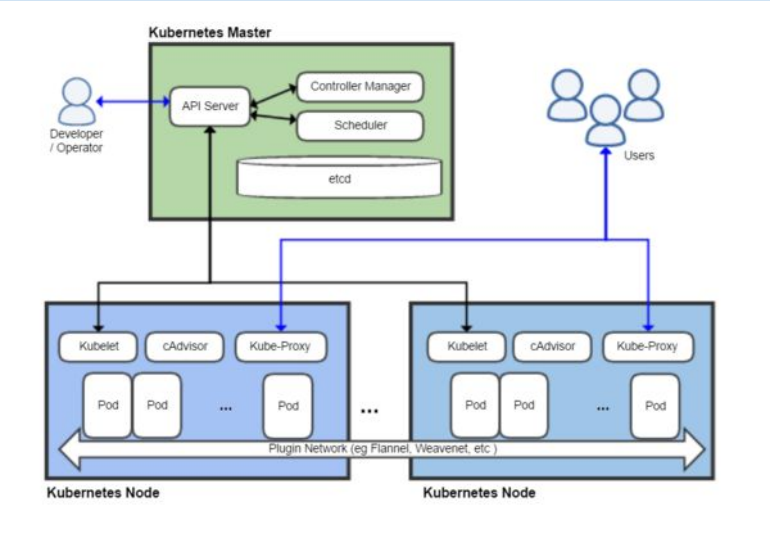
Características Generales:

- **Gestión de cambios** para aplicaciones en contenedores: Despliegues (*rollouts*) y recuperación de estados previos (*rollback*) de recursos distribuidos
- **Independiente** de proveedores, lenguajes/frameworks de desarrollo, herramientas de monitorización, registro y cualquier servicio adicional

Características k8s

- **Reubicación** automática de contenedores en un clúster de equipos anfitriones (nodos) según sus necesidades de recursos
- **Auto-escalado de contenedores** (pods) horizontal (HPA) y vertical (VPA)
- **Auto-escalado horizontal de nodos del Cluster**, es decir, VMs
- **Servicios integrados habituales en aplicaciones distribuidas**
  - **Descubrimiento de servicios** y **balanceo de carga**
  - Gestión de la **configuración** y **secretos**
  - **Gestión de almacenamiento** virtual
  - **Control de acceso a recursos**
  - Planificación de procesos por **lotes**
- **Admite extensión de funcionalidades mediante *plug-ins***

Arquitectura de Kubernetes



En **producción** debe haber al menos 2 nodos para garantizar HA

En entornos de **desarrollo** se puede trabajar con 1 solo nodo con herramientas como kind, minikube o Docker Kubernetes Engine,

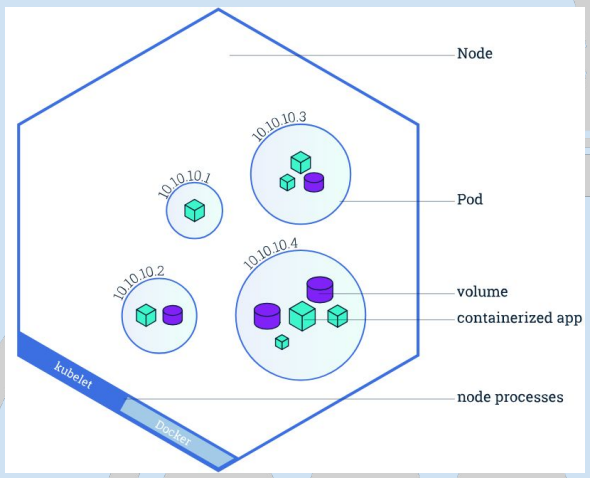
Fuente: wikipedia

Componentes principales k8s

- **Nodos,** máquinas físicas o virtuales sobre las que se ejecutan los componentes
- **Nodo/s Máster,** control del clúster, gestión de la carga de trabajo y comunicación entre elementos del sistema
  - **etcd**, repositorio clave-valor con información de configuración de **estado del clúster**
  - **Controller manager**, gestor de funcionamiento de despliegues
  - **API server,** para comunicación y gestión del cluster
  - **cAdvisor**, monitorización de recursos y rendimiento
- **Nodos esclavos**, ejecutan los contenedores
  - **Container runtime,** encargado de la ejecución del tipo de contenedores escogido, e.g. containerd, cri-o, docker engine,...
  - **Kubelet**, control del estado de ejecución de los contendores
  - **Kube-proxy**, con funciones de proxy y balanceo de carga para permitir acceso a los contenedores
  - **Otros elementos**: interfaz web (dashboard), autodescubrimiento (DNS), monitorización de métricas, e.g. cpu, memoria, disco, red… y registro, logging

Conceptos Kubernetes (I)

- **Pod**, agrupación de 1 o varios (*sidecars*) contenedores en una misma ubicación (**nodo**) que comparten recursos, e.g. ip, volumen, …
  - Unidad básica de planificación
  - No conservan el estado, descartables
  - Los contenedores en un mismo pod se comunican a través de localhost
- **Volúmenes**, Permiten a los contenedores acceder a recursos externos: discos, ficheros, variables de estado, secretos,... independientes de su ciclo de vida



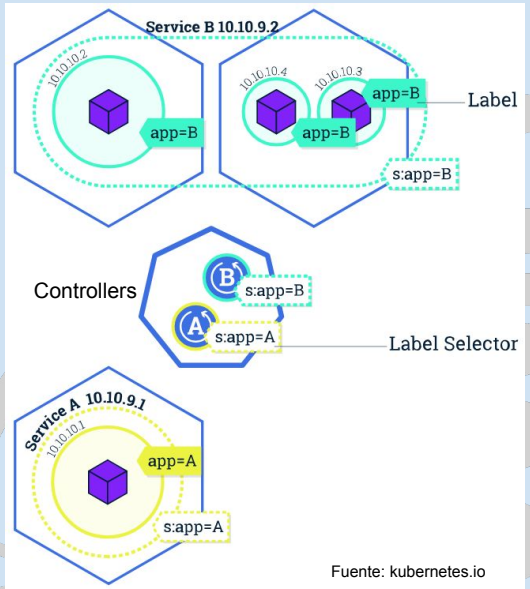
Fuente: kubernetes.io

Conceptos Kubernetes (II)

- **Controladores**, encargados de mantener o variar el estado de un sistema, e.g. despliegue de pods (*deployments*), controlador de replicación (*replica sets*), ...
- **Servicios**, conjunto variable de pods que trabajan conjuntamente. Tienen asignada una ip interna estática (o pública) y dns para su localización, con soporte de balanceo básico de carga.

Organización lógica de objetos

- **Namespace,** agrupación lógica de recursos en un mismo cluster, e.g. kube-system, default, front-end, back-end, etc.
- **Etiquetas y selectores**, clave=valor, asignadas a elementos para localización mediante selectores, e.g. identificar los pods de un servicio



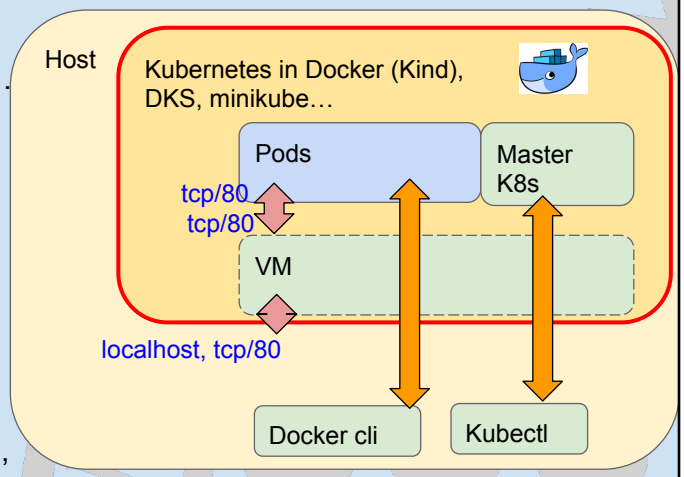
Fuente: kubernetes.io

Experimentando con Kubernetes

Universidad de Jaén

Requisitos:

- **Hypervisor**, e.g. hyper-v, hyperkit, virtualbox, kvm, … o **Runtime de contenedores**, e.g. Docker, Podman…
- **Instalación de kubernetes local:**
  - Instalación local con 1 solo nodo
  - e.g. docker kubernetes system (DKS), kind, minikube, microk8s, k3s ...
- **Kubectl**, cliente CLI para interacción con API K8s (incluido en dks)
- **Docker,** cliente en host, para interacción con servicio Docker en k8s, e.g. creación de imágenes



Primeros pasos con DKS

Universidad de Jaén

Activar Kubernetes desde *Preferencias* en Docker for Desktop
→ Seleccionar clúster basado en Kind con 1 o 2 nodos

Consultar apéndice sobre cómo trabajar en linux o cloud-shell sobre clúster local con minikube

Es conveniente utilizar como apoyo alguna herramienta visual como k9s, la extensión de kubernetes para vscode, …

comprobar conexión con servidor

```
$ kubectl version
```

Consultar clusters kubernetes administrados

```
$ kubectl config get-clusters
```

Conectar Kubectl con API cluster DKS y consultar nodos (sólo 1)

```
$ kubectl config use docker-desktop  #e.g. cambiar a cluster local o GKE
$ kubectl get nodes -o wide
```

Consultar pods

```
$ kubectl get pods -A # obtener pods en todos os namespaces
```

o usar herramientas externas, e.g. k9s, vscode con plugin k8s, …

Parar servicio k8s

Desde menú desplegable → kubernetes → disable local cluster

Control imperativo de K8s

Universidad de Jaén

Uso del cliente CLI **kubectl** para acceso del API K8s del cluster (-h para ayuda)

Creación de objetos

- **create** object_type, para crear cualquier tipo de objeto k8s, e.g. pods, servicios, etc.
- **create deployment**, crea un controlador *Deployment* para ejecutar uno o más pods
- **expose**, crear un objeto *Service* para permitir acceso a un conjunto de pods
- **autoscale**, crea un objeto para controlar el autoescalado horizontal (hpa) de un conjunto de pods.

Consulta de objetos

- **get** tipo objeto, información básica
- **describe**, información detallada
- **logs**, stdout/stderr de contenedores en un pod

¿Por qué no hay opciones de parar (*stop*) un pod?

Actualización de objetos

- **set,** modificar una propiedad de un objeto, e.g. imagen de un contenedor
- **label**, gestionar etiquetas de un objeto
- **scale**, actualizar manualmente el número de réplicas de un *Deployment*
- **edit**, utilizar editor de textos para modificar objeto k8s

Borrado de objetos

- **delete** tipo_objeto/nombre_objeto

CC BY SA

Ejemplos de comandos básicos

Universidad de Jaén

Lanzar un contenedor en un nuevo POD (un *deployment* controlará la ejecución) y publicar en POD puerto del contenedor

```
$ kubectl create deployment --image=traefik/whoami wwwserver
```

Publicar el puerto del pod en el Nodo (Creando un servicio)

```
$ kubectl expose deployment wwwserver --port=80 --type=NodePort
```

Consultar elementos creados:

```
$ kubectl describe service wwwserver
$ kubectl get pods -o wide       #nodes, deployments, services… usar all para listar todos
$ kubectl logs svc/wwwserver     #consultar logs
$ kubectl exec wwwserver-6d4977554c-8glrd -it -- bash # ejecutar comandos en pod
```

Consultar servicios publicados en cluster (internos o externos)

```
$ kubectl get services
```

Parar recursos: Servicio, Deployment y Pod

```
$ kubectl delete service,deployment wwwserver
```

+info en Kubectl for docker users

CC BY SA

Generando y lanzando contenedores con DKS

Acceso a imágenes de contenedores:

- un cluster kubernetes no dispone de un registro de imágenes integrado
  - … pero como el cluster DKS está integrado en la instalación de Docker Desktop tiene acceso a las imágenes en el registro local para lanzarlas en pods
  - En un cluster Kind, las imágenes locales deben enviarse al clúster, e.g. *kind load …*
- Por defecto las imágenes se descargan de docker hub https://hub.docker.com

Trabajar con proyecto Docker normalmente, todas las operaciones se realizan en gestor Docker de DKS

```
$ cd; git clone https://github.com/jrbalsas/asnwebapps
$ cd asnwebapps/phpwebapp
$ docker build . -t asn/phpwebapp:v1
$ docker images  # consultar imágenes locales docker
```

IMPORTANTE: añadir/etiquetar versión de la imagen para que K8s utilice la imagen local si está disponible. En otro caso tratará de descargarla de un registro remoto

Lanzar contenedor mediante un deployment

```
$ kubectl create deployment --image=asn/phpwebapp:v1 phpwebapp
```

Crear servicio y publicar puerto

```
$ kubectl expose deployment phpwebapp --port=80 --type=NodePort  # --target-port=80
$ kubectl get svc  # consultar puerto publicado y acceder en localhost, e.g. 80:32310/TCP
```


Escalado y auto-escalado horizontal

- Los controladores de escalado se encargan de mantener el estado de los PODS
- Las réplicas de pods en un *deployment* pueden ajustarse de dos formas:

Escalado manual

Monitorizar réplicas (con k9s o kubectl en ventana aparte)

```
(shell 1) $ kubectl get pods -w --selector app=phpwebapp # ojo, no parar comando
(shell 2) $ kubectl scale --replicas=3 deployment phpwebapp
```

Auto-escalado (horizontal)

- Basado en utilización de cpu o incluso métricas personalizadas
- Creación de objetos específicos *HorizontalPodAutoscales (hpa)*

```
$ kubectl autoscale deployment phpwebapp --min=2 --max=5 --cpu-percent=80
```

Infrastructure as a Code, en K8s

Motivación: definir la infraestructura de una aplicación en la nube mediante ficheros que definen sus componentes, características e interacciones

Ventajas:

- **Replicación automática** de infraestructuras complejas sin necesidad de procesos manuales
- **Control de cambios** mediante sistemas de control de versiones, e.g. git
- **Análisis** estático de características de infraestructura, e.g. aspectos de seguridad

IaaC en K8s

- **Definición y estado** de objetos mediante objetos JSON
  - Edición más simple en formato jerárquico YAML
- **Gestión de objetos mediante ficheros**
  - **Imperativa**
    - `$ kubectl create/replace/delete -f fichero_dir/ [-R]`
    - `$ kubectl get tipo objeto -o yml, # obtener estado actual del objeto`
    - `$ kubectl edit tipo objeto # Editar y aplicar nuevo estado`
  - **Declarativa**
    - Enfoque preferido
    - K8s adapta el estado del objeto para reflejar una nueva configuración
    - `$ kubectl apply -f fichero_dir/`

Gestión de ciclo de vida de Pods

Los controladores definen ciclo de vida de los pods según sus características:

- **Deployments,** gestionan el número y tipo de instancias activas (*ReplicasSet*) además de permitir recuperación de un estado previo (*rollback*)
  - Estrategias de despliegue:
    - *Recreate*, destruir las instancias activas y lanzar las nuevas
    - *RollingUpdate*, se van apagando progresivamente las instancias actuales y se van lanzando las nuevas, e.g. nueva versión de la app
- **StatefulSets**, control de pods con identidad de red permanente y activación/desactivación ordenada, e.g. databases
- **DaemonSets**, control de pods con réplica en cada nodo, e.g. monitorización de nodos...
- **Job, CronJob**, Ejecución de tareas por lotes o programadas

Ejemplo de despliegue declarativo

Ejemplo: *frontend-deployment.yml*

```
apiVersion: apps/v1
kind: Deployment                Elemento k8s
metadata:
  name: frontend-deployment     Identificador del controlador de despliegue
spec:
  selector:
    matchLabels:
      app: phpwebapp            Identificación del los pods gestionados
  replicas: 2                   Número de pods activos que hay que mantener
  template:                     Descripción de los PODS gestionados
    metadata:
      labels:
        app: phpwebapp              etiquetas de los pods
    spec:
      containers:                   Características del contenedor en cada pod
      - name: phpwebapp
        image: asn/phpwebapp:v1
          imagePullPolicy: IfNotPresent Utilizar imagen local por defecto
        ports:
        - containerPort: 80
```

```
$ kubectl create -f frontend-deployment.yml
$ kubectl get deployments
$ kubectl get pods --selector app=phpwebapp
$ kubectl describe deployments frontend-deployment
Incluso podemos hacer cambios directos en el objeto...
$ kubectl edit deployment frontend-deployment
```


Servicios

Solución al problema de **autodescubrimiento** de pods dinámicos: los pods de un despliegue son descartables y su ip no debe utilizarse para contactar con ellos

- Los **servicios** definen una capa de abstracción que representa a una colección de pods y establecen una forma de acceder a ellos, e.g. ip, balanceo, etc.
- **Métodos de localización en cluster:**
  - Variables de entorno en PODS, e.g. WWW_FRONTEND_SERVICE_HOST=10.0.0.10, WWW_FRONTEND_SERVICE_PORT=80
  - Si está activo el **componente de DNS**, accesibles desde Pods usando su nombre como host, e.g. www-frontend

Ejemplo:

```
kind: Service
apiVersion: v1
metadata:
  name: www-frontend  #service dns name
spec:
  selector:
    app: phpwebapp #Pods gestionados
  ports:
  - protocol: TCP
    port: 80 #puerto del servicio en nodos (interno)
    targetPort: 80 #puerto en pods (opcional)
```

Tipos de Servicio

- **ClusterIP** (por defecto), accesible en IP interna exclusivamente dentro del clúster o sin ip (None)
- **NodePort**, igual que *ClusterIP* para acceso dentro del clúster, pero también permite acceso externo en otro puerto vinculado a la ip externa de los nodos.
  - Restricciones: la ip de los nodos puede variar, el puerto externo está en el rango 30000-32767
- **LoadBalancer**,
  - Expone el servicio a través de un balanceador de carga del proveedor en una ip y puertos fijos para redes externas
  - Crea automáticamente un *ClusterIP* y un *NodePort. Requiere costes adicionales por servicio (ip estática por servicio)*
- **ExternalName**, vincula el servicio a una máquina externa al cluster para acceso unificado desde todos los pods del cluster

Ejemplo de Servicio público

Ejemplo:

```
kind: Service
apiVersion: v1
metadata:
  name: www-frontend
spec:
  selector:
    app: phpwebapp  #Pods accesibles por servicio
  ports:
  - name: http
    protocol: TCP
    port: 80            #puerto privado del servicio en clúster
    targetPort: 80      #puerto en pods
    nodePort:   30000   #Puerto público (se suele omitir para valor aleatorio)
  type: NodePort        #Publicar puerto en todos los Nodos
```

Problemas:

- La ip pública de cualquier nodo permite acceso al servicio, pero las ips de los nodos pueden variar, no estar accesibles, …
- No hay mucho control sobre cómo balancear la carga entre los Pods
- Puertos públicos limitados, e.g. 30000-32767

Modalidades de acceso externo a servicios



- **Servicio de tipo LoadBalancer**
  - ClusterIP que delega en un balanceador **externo al cluster**
  - Balanceo TCP/UDP (L4)
  - Sin terminación de canal SSL
  - Proporcionado por proveedor. Costes por cada servicio que lo utilice
- **Recurso *Ingress***
  - Enrutador de peticiones HTTP(L7) externas a servicios internos del clúster
  - Suele recibir peticiones de un *LoadBalancer* externo
  - Requiere un software específico *(Ingress controller*) que se encargue de su gestión, e.g. nginx, haproxy, traefik, etc.
- **Recursos Gateway API**
  - Evolución de los *Ingress* que combina otros tipos de enrutado, e.g. L7 y L4
  - Admite y unifica configuraciones avanzadas de enrutamiento y funcionalidades adicionales de balanceo que previamente se debían configurar de forma independiente
  - Actualmente está en proceso de implantación (debe habilitarse expresamente) y está siendo adaptado por los diferentes desarrolladores a sus productos, e.g. traefik

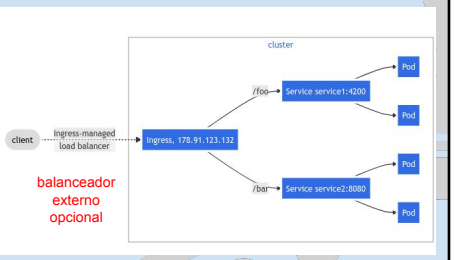


Acceso web a servicios en K8s: Ingress



Recurso *Ingress*

- **Características:**
  - Enrutamiento de tráfico L7 (HTTP/S) hacia servicios
  - Suele utilizar un software balanceador **interno al clúster**: nginx, haproxy, traefik, Gke ingress (GLBC), …
  - Gestión de conexiones TLS, certificados…
  - Enrutado mediante peticiones por nombre de host y rutas (path) en peticiones HTTP
  - Otras características propias de la implementación utilizada o plugins adicionales, e.g. cert-manager, integración con balanceador externo, control de carga, etc.
- Configuración más compleja que un servicio LoadBalancer:
  - Instalación de software específico en cluster y configuración propietaria para características adicionales, e.g. control de carga, detección de nodos caídos, etc.
- Costes: como recursos del cluster (implementación específica) o según proveedor (e.g. conectado a GLBC) o



Fuente: kubernetes.io



Ejemplo de Servicio Balanceado externamente

```yaml
kind: Service
apiVersion: v1
metadata:
  name: www-frontend
spec:
  selector:
    app: phpwebapp
  ports:
  - name: http
    protocol: TCP
    port: 80            # puerto privado del servicio en cluster
    targetPort: 80      # puerto en pods
  type: LoadBalancer    # Habilitar acceso desde LB externo
#  loadBalancerIP: "YOUR.IP.ADDRESS.HERE"    # opcional según proveedor
```


Ejemplo de enrutado con Ingress

```yaml
kind: Service
apiVersion: v1
metadata:
  name: www-frontend
spec:
  selector:
    app: phpwebapp
  type: ClusterIP
  ports:
  - name: http
    protocol: TCP
    port: 80
    targetPort: 80
```

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: lb-ingress
spec:
  defaultBackend: # Backend por defecto
    service:
      name: www-frontend
      port:
        number: 80
  rules:                # Reglas opcionales de enrutamiento
    - host: myhost1.com
      http:
        paths:
        - path: /
          pathType: Prefix
          backend:
            service:
              name: www-frontend_2   # Otros servicios del clúster
              port:
                number: 8080
#   - host: myhost2.com
#   tls:                # Configuración terminación TLS...
#   - hosts:
#     - myhost1.com
#     secretName: testsecret-tls
```

Segmentación de tráfico en K8s

Recursos *Network Policy*

- Definen restricciones de tráfico entrante (*ingress*) o saliente (*egress*) entre pods de un despliegue
- Requieren un software específico instalado en el clúster: CNI (Container Network Interface), e.g. Calico, Flannel, Istio, …
- **Consideraciones de uso**
  - Por defecto un pod no tiene restricciones de tráfico
  - Si se define una política de entrada o salida para un pod, se deniega lo que no se indique expresamente
  - Si varias reglas se aplican a un mismo pod lo hacen en modalidad OR: ¡**OJO!** no se puede limitar algo que otra regla haya autorizado → Mejor denegar por defecto y autorizar expresamente

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: wwwserver-access-policy
spec:
 podSelector:
   matchLabels:
     app: phpwebapp  # target pods
 ingress: # incoming traffic allowed
 - from: # only from selected pods
   - podSelector:
       matchLabels:
         role: bastion
   ports:  # destination ports
     - protocol: TCP
       port: 22
 - ports: # from any host/pod
   - protocol: TCP
     port: 80
   - protocol: TCP
     port: 443
 egress: # outgoing traffic allowed
 - to: # only to selected pods
   - podSelector:
       matchLabels:
         role: database
   ports:
     - protocol: TCP
       port: 3306
```


Otros recursos en K8n

Consultar algunos detalles de uso en anexos

Elementos habituales en configuraciones en la nube

- **ConfigMaps**, acceso a elementos de configuración mediante variables de entorno o ficheros
- **Secrets,** acceso a datos sensibles, tokens, credenciales, certificados. Solo disponibles en memoria.
- **Volumes Persistentes,** externos o internos
- **Control de acceso** a operaciones en clúster (RBAC)
  - *Role, ClusterRole*
    - Definen permisos de acceso a operaciones en un *namespace* o en el cluster completo, e.g. crear o parar pods, etc.
  - *RoleBindings*
    - *Asignación de roles a usuarios, grupos o servicios*

Conclusiones

- Las **VPCs** y las **reglas de cortafuegos** permiten organización y control de acceso de componentes de una app en la nube
- Los **sistemas de orquestación** permiten el diseño sistematizado, despliegue y gestión automatizada de aplicaciones distribuidas en la nube
- El **diseño declarativo de infraestructuras distribuidas o IaaC** permite su configuración, gestión, monitorización y control de cambios utilizando sistemas de control de versiones de código
- Sistemas como **Kubernetes** permiten abstraer los recursos y servicios heterogéneos por distintos proveedores en la nube
- **Kubernetes incorpora por defecto funcionalidades para el uso de patrones de diseño en la nube** como balanceo de carga, auto-descubrimiento y configuración de servicios, gestión de volúmenes distribuidos, etc. sin necesidad de recurrir a servicios externos

Referencias

- Chris Dotson, “Network Security”*, in Practical Cloud Security,* 1st edition. O’Reilly Media, Inc, 2019 [on-line]. Disponible en recurso digital UJA
- *Kubernetes Documentation.* Kubernetes,2026 [on-line]. Disponible en https://kubernetes.io/docs
- *Learn Kubernetes Basics.* Kubernetes, 2026 [on-line]. Disponible en *https://kubernetes.io/docs/tutorials/kubernetes-basics/*
- *Kubernetes Tutorial*. Tutorialspoint, 2026 [on-line]. Disponible en https://www.tutorialspoint.com/kubernetes
- *kubectl Cheat Sheet*. Kubernetes, 2026 [on-line]. Disponible en https://kubernetes.io/docs/reference/kubectl/cheatsheet/
- *Deploying a containerized web application*. Google Cloud, 2026 [on-line]. Disponible en https://cloud.google.com/kubernetes-engine/docs/tutorials/hello-app
- *Overview of Cluster Administration*. Kubernetes Engine. Google Cloud, 2026 [on-line]. Disponible en https://cloud.google.com/kubernetes-engine/docs/how-to/cluster-admin-overview

Anexos

- Segmentación de red en GCP
- Otros recursos en K8S
- Uso básico de minikube
- Gestión de un clúster K8S con GKE

Configuración de redes privadas en GCP

Segmentación de VPCs

Segmentación en GCP

Google Cloud Platform

Universidad de Jaén

VPC

- Automáticas (por zonas en rango 10.128.0.0/9) o manuales (resto de rangos privados)
- Reglas de enrutado entre subredes internas (automáticas o manuales)

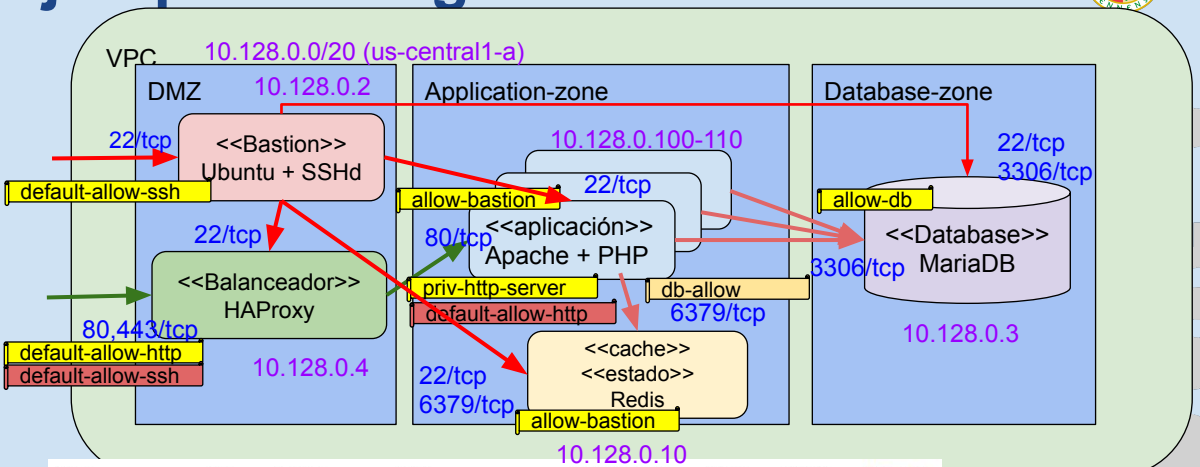
Reglas de cortafuegos

- Por VPC, tráfico IPv4, pueden deshabilitarse temporalmente
- Reglas por defecto para tráfico http, https, ssh, rdp, icmp (ping), ~~25~~
- **Prioridad**: 1 más alta, 65535 más baja
- **Acción**: allow / deny
- **Objetivo**, instancias a las que se aplica la regla: todas, *etiquetadas* o servicio
- **Sentido**: ingress+origen / egress+destino
- **Origen o Destino**: IP + puerto
- Opcional: habilitar **logs de acceso** (ojo con volumen de datos y costes)

CC BY SA

Ejemplo de segmentación en GCP

Universidad de Jaén

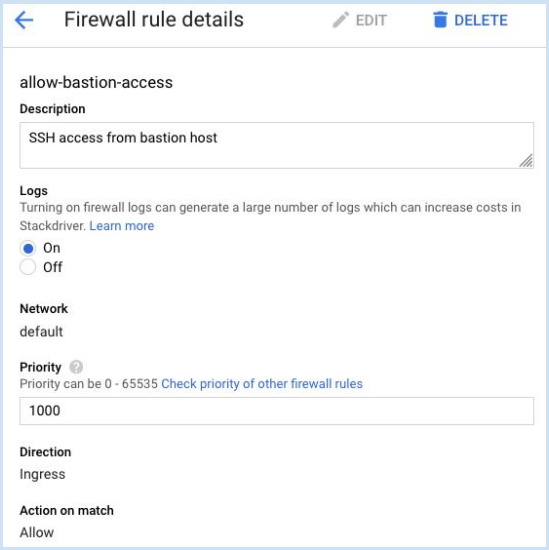


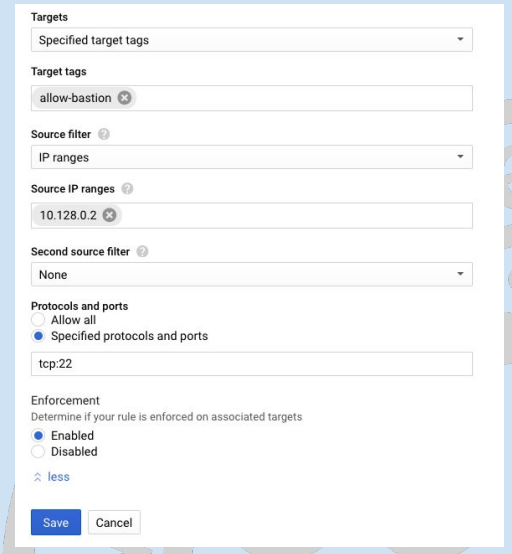
Name	Type	Targets	Filters	Protocols / ports	Action	Priority	Network
db-access	Egress	db-allow	IP ranges: 10.128.0.3/32	tcp:3306	Allow	1000	default
allow-bastion-access	Ingress	allow-bastion	IP ranges: 10.128.0.2	tcp:22	Allow	1000	default
private-http-access	Ingress	priv-http-server	IP ranges: 10.128.0.4	tcp:80	Allow	1000	default

tráfico entrante
tráfico saliente
tráfico bloqueado

CC BY SA

Edición de reglas de cortafuegos





Edición de reglas de cortafuegos

```
gcloud compute firewall-rules create [NAME] \
    [--network [NETWORK]; default="default"] \
    [--priority [PRIORITY];default=1000] \
    [--direction (ingress|egress|in|out); default="ingress"] \
    [--action (deny | allow )] \
    [--target-tags [TAG][,TAG,...]] \
    [--target-service-accounts=[IAM Service Account] \
    [--source-ranges [CIDR-RANGE][,CIDR-RANGE…]] \
    [--source-tags [TAG][,TAG,...]] \
    [--source-service-accounts=[IAM Service Account] \
    [--destination-ranges [CIDR-RANGE][,CIDR-RANGE...]] \
    [--rules ([PROTOCOL][:PORT[-PORT]],[PROTOCOL[:PORT[-PORT]],...]] | all ) \
    [--disabled | --no-disabled]
    [--enable-logging | --no-enable-logging] (requires `gcloud beta`)
```

```
$ gcloud init                              # Inicializar conexión, seleccionar proyecto y zona

$ gcloud compute firewall-rules list       #Listar reglas en proyecto actual

$ gcloud compute firewall-rules create allow-bastion-access \

     --direction ingress --target-tags allow-bastion --source-ranges=10.128.0.2 \

     --rules tcp:22 --action allow --enable-logging

$ gcloud compute firewall-rules update allow-bastion-access --no-enable-logging
```

Otros recursos Kubernetes

Configuración de recursos adicionales

Auto-configuración de Pods

Objetivos:

- permitir a los contenedores obtener información del cluster al iniciarse para definir su comportamiento, e.g. ficheros de configuración, credenciales, certificados,...
- centralizar detalles de configuración en clúster

Métodos de paso de información:

- *Variables de entorno* en el SO del contenedor
- *Ficheros* con detalles de configuración en sistema de archivos

Recursos k8s:

- **ConfigMap**, recurso para referenciar elementos de configuración:
- **Secrets,** similar al anterior con mejoras de seguridad e.g. ofuscamiento de datos, almacenamiento en memoria..

Ejemplos de configMap

Universidad de Jaén

Ubicar ficheros de configuración y/o ficheros de propiedades e.g. php.ini, env.properties...

```
$ mkdir -p appconfig
$ echo -n "#Sample php.ini" > appconfig/php.ini
$ echo -n "db.host=dbserver" > appconfig/env.properties

$ kubectl create configmap phpapp-config --from-file=appconfig/php.ini
$ kubectl create configmap phpapp-env  --from-env-file=appconfig/env.properties
$ kubectl describe configmaps phpapp-config phpapp-env  #ver contenidos
```

Ejemplos de uso de configuración en variables de entorno y ficheros de configuración

```
...
spec:
  containers:
  - image: asn/phpwebapp
    imagePullPolicy: IfNotPresent
    name: phpwebapp
    ports:
    - containerPort: 80
      protocol: TCP
    env:
    - name: DB_HOST   #variable de entorno
      valueFrom:
        configMapKeyRef:
          name: phpapp-env
          key: db.host    #clave con valor en fichero de variables
    volumeMounts:
    - mountPath: /usr/local/etc/php   #ubicación ficheros configuración
      name: config-files  #volumen con fichero configuración
  volumes:
      ...
```

```
volumes:
  - name: config-files
    configMap:
      name: phpapp-config
      items:
        - key: php.ini
          path: php.ini #fichero en destino
```


CC BY SA

Secretos en k8s

Universidad de Jaén

- Utilizar para información sensible de: claves, tokens, claves tls, …
- Se almacenan en memoria de nodos o, si el cluster necesita almacenarlos, lo hace de forma cifrada
- Ofuscados Base64, solo pueden consultarlos los PODs preestablecidos, se puede restringir su acceso a usuarios del clúster
  - ***Recomendación:*** *si se quiere evitar el acceso a un secreto de un pod, crear un contenedor que utilice el secreto y otro expuesto que interaccione con él.*
- **Tipos:**
  - ***generic***, contenidos para pods
  - ***tls***, específico para distribución de certificados
  - ***docker-registry***, para credenciales de acceso a registro de contenedores privado

CC BY SA

Ejemplo de uso de secretos

Uso similar a configMap

```
$ cd appconfig
$ echo -n
'db.user=wwwuser\ndb.password=892r8323wo83d' >
dbcredentials.txt

$ kubectl create generic secret db-credentials
--from-env-file=dbcredentials.txt

$ kubectl get secrets
$ kubectl describe  secrets db-credentials
```

Ejemplo de uso de secretos en variables de entorno

```
...
spec:
  containers:
  - image: asn/phpwebapp
    imagePullPolicy: IfNotPresent
    name: phpwebapp
    ports:
    - containerPort: 80
      protocol: TCP
    env:
    - name: DB_USER       #variable de entorno
      valueFrom:
        secretKeyRef:
          name: db-credentials
          key: db.user    #clave con valor en secreto
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-credentials
          key: db.password
```


Volúmenes persistentes en K8s

Problema:
las imágenes de un contenedor no son persistentes. ¿Cómo obtener unidades de almacenamiento persistente?

Solución: Volúmenes en k8s

- Vinculados a un pod. Persisten entre reinicios de contenedores en el pod
- k8s soporta varios **tipos de volúmenes**:
  - **Locales al nodo**:
    - ***emptyDir***, permanente entre reinicio de un contenedor
    - ***hostPath***, montaje de recursos en Nodo (ojo si pod se mueve)
  - **Específicos para configuración**: ***configMap*** y ***secret***
  - Según el proveedor, e.g. *AWS ebs, Azure Disk,* ***gcePersistentDisk****,...*
  - **En red**, e.g. *nfs, smb, …*

Ejemplo de uso de volúmenes

```
spec:
  containers:
  - name: myapp
    image: myappimage
    volumeMounts:
    - mountPath: /data1
      name: local-volume
    - mountPath: /data2
      name: data-volume
  volumes:
    -name: local-volume
     hostPath:                # Nodo Minikube, carpeta local usuario
       path: /c/Users/Paco/phpwebapp/data
    -name: data-volume
     gcePersistentDisk:   # persistente en cluster
       pdName: app-data-disk #id volumen definido en GCE
       fsType: ext4
```


Declaración y uso de Volúmenes

Objetivo: Separar declaración de volúmenes requeridos de un Pod de sus características y ubicación definitivas

- **Conceptos:**
  - ***PersistentVolume*(PV),** volumen de almacenamiento definido por un administrador. Pueden ser estáticos o creados dinámicamente según demanda.
    - Varios tipos: GCEPeristentDisk, AWSElasticBlockStore, AzureFile, AzureDisk, HostPath (solo pruebas en local), …
    - Los volúmenes dinámicos se reservan a partir de diferentes tipos definidos mediante *StorageClass* por el administrador del clúster
  - ***PersistentVolumeClaims*(PVC)**, solicitud de almacenamiento de un usuario sobre un PV: espacio solicitado y tipo de acceso, e.g. RW. Un pod solicita un volumen indicando un PVC.
    - Debe coincidir con algún PV o debe haber recursos en la clase de almacenamiento solicitada
- Son objetos k8s, `$ kubectl get persistentvolumes o persistentvolumeclaims`

Ejemplo de declaración y uso de volúmenes




```yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: www-volumeclaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-deployment
spec:
  selector:
    matchLabels:
      app: phpwebapp
  replicas: 2
  template:
    metadata:
      labels:
        app: phpwebapp
    spec:
      containers:
      - name:
        image: asn/phpwebapp
        ports:
          - containerPort: 80
        volumeMounts:
          - name: www-persistent-storage
            mountPath: /var/html/www/data
      volumes:
      - name: www-persistent-storage
        persistentVolumeClaim:
          claimName: www-volumeclaim
```

- Dejamos que el Clúster utilice la clase de almacenamiento que tenga por defecto
- Ventaja de separar declaración y uso: podemos cambiar la ubicación del volumen de datos dependiendo del tipo de despliegue. e.g. GKE, Minikube





Minikube

Guía básica de uso

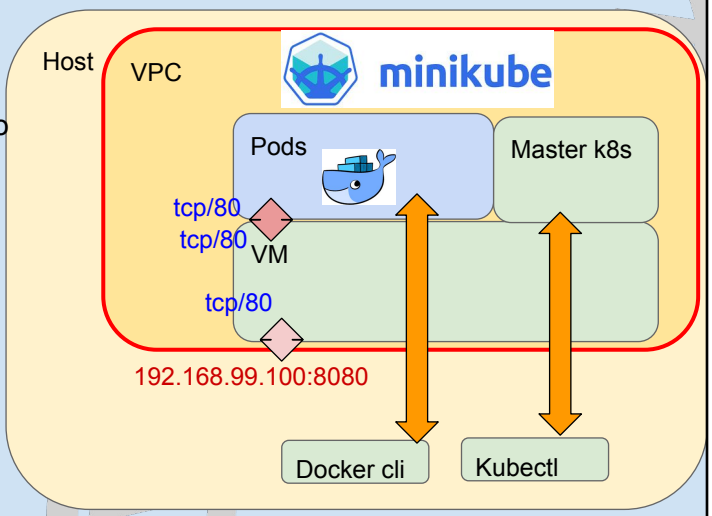




Experimentando con Kubernetes

Requerimientos:

- **Docker o Hypervisor L1**, e.g. virtualbox, kvm
- **Minikube,**
  - Multiplataforma
  - Instalación de k8s local con 1 solo nodo para experimentación
  - Incluye servicios k8s
  - Puede lanzarse incluso dentro de Docker
- **Kubectl**, cliente CLI para interacción con API k8s (similar a cliente CLI docker)
- **Docker** en host, para interacción con servicio Docker en k8s



Minikube, primeros pasos

Inicializar cluster Minikube (crear VM en primer lanzamiento)

```
$ minikube start   #se puede especificar hypervisor, e.g virtualbox, hyperv, …)
```

Consultar estado

```
$ minikube status
$ minikube ip      #Obtener ip nodo en vpn cluster
```

Conectar Kubectl con API cluster minikube

```
$ kubectl config use minikube
$ kubectl get nodes
```

Conectar con VM minikube y consultar contenedores...

```
$ minikube ssh
$ docker ps
```

Lanzar consola gráfica del cluster (en terminal aparte)

```
$ minikube dashboard
```

Parar VM cluster

```
$ minikube stop
```

Generando y lanzando contenedores en minikube

Universidad de Jaén

Es necesario **conectar el cliente docker CLI local con servidor docker en VM Minikube**

```
$ eval $(minikube docker-env)  #repetir con docker-env -u para desconectar
$ docker ps  # muestra contenedores en VM minikube
```

Trabajar con proyecto Docker de forma habitual, todas las operaciones se realizan en gestor Docker de Minikube

```
$ cd; git clone https://github.com/jrbalsas/asnwebapps
$ cd asnwebapps/phpwebapp
$ docker build . -t asn/phpwebapp:v1 # indicar versión para dar prioridad a despliegue con imagen local
$ docker images  # imágenes locales docker en Minikube
```

Lanzar contenedor (forzamos a utilizar la imagen local creada o usar imagen en docker hub)

```
$ kubectl create deployment --image=asn/phpwebapp:v1 phpwebapp
```

Crear servicio y publicar puerto

```
$ kubectl expose deployment phpwebapp --port=80 --type=NodePort
$ minikube service phpwebapp  # abrir url del servicio
```


CC BY SA

Universidad de Jaén

Gestión de un cluster K8s

Google Kubernetes Engine, GKE

CC BY SA

Google Kubernetes Engine, GKE

Universidad de Jaén

Características

- **No es necesario nodo máster**, GKE se encarga de sus labores
- Uso de **GLBC**, servicio de balanceador de carga de GCP
- Creación de **pools de nodos** con características diferentes en un mismo cluster
- **Escalado automático de nodos** del cluster según demanda
- **Actualización automática** del software de los nodos
- **Auto-reparación de nodos** para garantizar la disponibilidad
- **Registro y monitorización centralizados** con servicio *Stackdriver* de GC
- *GKE marketplace* para instalación de aplicaciones habituales en la nube

CC BY SA

Creación de un clúster GKE

Universidad de Jaén

- Para optimizar los recursos, seleccionar el modo **Estándar** para una administración completa de las características del clúster



Configurar los siguientes elementos:

- **Aspectos básicos:**
  - Nombre del cluster, e.g. asn-cluster,
  - Tipo de ubicación: Zonal: elegir solo una zona e.g. us-central1-a, Recordar para conectar con el cluster más adelante
  - Ubicaciones predeterminadas de los nodo: Predeterminada
- **Configuración avanzada,** *Features, Cloud Monitoring*
  - marcar solo: volumes, pods, implement., daemonset, cadvisor y kubelet
- **Redes**:
  - Container Network Interface (CNI), habilitar Dataplane V2 (para usar *NetworkPolicies*)
- **Grupo de nodos:** Reducir el número y tamaño de los nodos
  - Default-pool, cantidad de nodos, reducir a 1
  - Tipo de nodo: e2-medium (2vcpu, 4 gb ram)

Los ajustes permitirán reducir el coste por defecto de más de 300$/mes a una tercera parte.

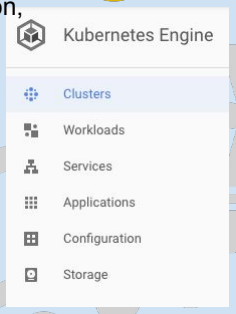
El proceso de creación puede llevar unos minutos hasta completarse

Desde las opciones del cluster es posible modificar algunas opciones, consultar el estado de los nodos o revisar los logs del cluster entre otras

CC BY SA

Gestión de un cluster GKE

La interfaz gráfica permite la consulta y operaciones básicas sobre objetos del cluster: edición, borrado, etc.



Acceso CLI (local o GCP shell)

en debian/ubuntu es necesario instalar los paquetes google-cloud-cli y google-cloud-cli-gke-gcloud-auth-plugin

Configurar acceso con kubectl a un cluster creado en consola o GCE Shell

```
$ gcloud auth login
$ gcloud container clusters get-credentials asn-cluster
```

(puede ser necesario añadir la zona y proyecto, e.g. --zone us-central1-a --project=ASN2026_…)

Cambiar entre clusters disponibles (cluster en GKE, docker desktop, …) para pruebas en producción o local

```
$ kubectl config get-contexts
$ kubectl config use-context  gke_asn-project_us-central1-a_asn-cluster
```

o volver a usar el cluster local, e.g. docker, minikube, …

```
$ kubectl config use-context docker-desktop
```


Despliegue de aplicaciones en GKE

- **Realizar operaciones habituales con kubectl** imperativas o declarativas… Algunos ejemplos simples:

```
$ kubectl create deployment --image traefik/whoami --port 80 testapp
```

- **Las imágenes propias publicarlas en un registro,** e.g. google cloud registry, gcr.io, docker-hub, ...
- **Publicar servicio en Internet y obtener ip pública** (ojo, se activará GLBC $$$)

```
$ kubectl expose deployment testapp --target-port=80 --port 80 --type=LoadBalancer
$ kubectl get service testapp --watch #esperar 2' a 5' hasta que muestre una ip externa de acceso
```

- **Nota:** Si no usáramos el balanceador, en GKE los **servicios NodePort no son accesibles desde el exterior** por defecto:
  - Los nodos GKE tienen ip en una VPC privada
  - Es necesario crear reglas específicas en firewall para darle acceso
  - No obstante se podría acceder para pruebas desde host local creando un proxy temporal

```
$ kubectl port-forward service/testapp 8080:80  #acceder a app en http://localhost:8080
```

Liberar recursos GKE

IMPORTANTE: Vigilar el coste de los recursos utilizados por el cluster usando la herramienta de facturación en el menú de servicios

- Durante las pruebas, los **servicios de tipo *LoadBalancer*** deben **destruirse para desactivar GLBC** y gastos derivados. Pueden volver a crearse cuando sean necesarios
  - Desde **consola web** o usando **kubectl**

```
$ kubectl delete service,deployment testapp
```

- Borrado del cluster

```
$ gcloud container clusters delete asn-cluster
```